

From Job Ads to Structured Skills

A two-stage pipeline with BERT and a Large Language Model

Prof.Dr. Marcel Blattner

Co-Head Applied AI Research Lab HSLU

`marcel.blattner@hslu.ch`

Why This Project Matters

Job ads are a goldmine of labor-market information - locked behind unstructured text.

- Recruiters, job seekers and labor-market analysts all need the same things: which skills, which qualifications, which benefits
- But the text is messy: free-form, mixed sections, varying length, no schema
- Reading thousands by hand is not an option - we need to extract structure automatically

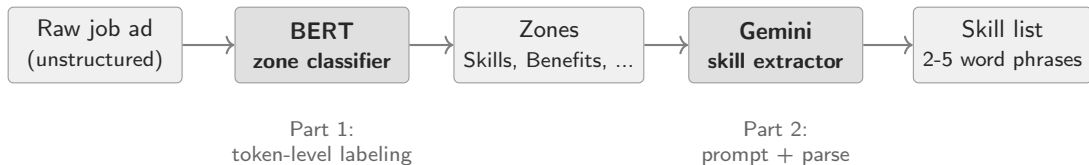
Our goal in one sentence

Turn raw job advertisements into a clean, queryable list of skills - automatically.

This project gives you hands-on experience with the two workhorses of modern NLP: a fine-tuned transformer (BERT) and a general-purpose LLM (Gemini). You will see where each one shines.

The Two-Stage Pipeline

One job ad goes in. A list of skills comes out. Two models do the work.



Why two stages?

- BERT is fast, cheap, and great at *structured* sequence tagging once trained
- LLMs are great at *open-ended* extraction but expensive to call on full ads
- Use BERT to find the small relevant chunk, then spend LLM tokens only there

The Dataset: annotated.json

Job advertisements with manually labeled spans of text - your only ground truth.

one job ad entry

`data['content_clean']`

cleaned full text of the ad

`annotations[0]['result']`

list of labeled spans, each with:

`value['start'], value['end']`

character offsets in the text

`value['text']`

the actual annotated substring

`value['labels'] = ["Abschlüsse"]`

the zone label assigned to the span

Note on permissions: the dataset is for restricted use within this student project - do not distribute it.

The Zones We Want to Identify

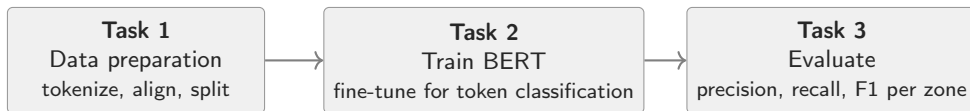
Each token gets exactly one of these labels - that is the prediction task.

Zone label	What it covers
Fähigkeiten und Inhalte	Skills and core content of the role - input for stage 2
Abschlüsse	Required degrees, certificates, formal qualifications
Erfahrung	Years of experience, prior roles, domain background
Anstellung	Employment terms - workload, location, contract type
Benefits	Perks, salary range, training, work culture
Firmenbeschreibung	The company itself - what it does, its size, its values
Bewerbungsprozess	How and where to apply, contact person, deadlines
O	Outside any defined zone - the default “background” label

Why “O” matters: most tokens in a typical ad will be “O”. The class imbalance is large and we will need to deal with it (see Task 2).

Part 1: Zone Identification with BERT

Three sequential tasks. The output of each is the input to the next.



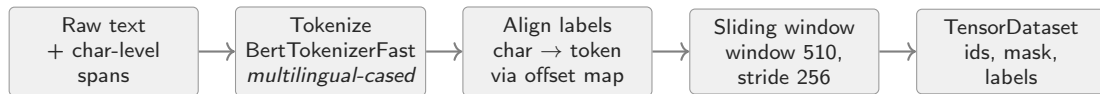
What you get from us

- `preprocessing.py` - a working data preparation script (read it carefully)
- `training_template.py` - a training scaffold with # TODO sections to fill in
- Two support PDFs explaining what each script does

Your job: understand both scripts, complete the TODOs, train the model, evaluate properly.

Task 1 - Data Preparation

Character-level annotations have to become token-level labels. That is the whole game.



Three things to be careful about

- **Subword tokens.** “Bewerbungsprozess” becomes several tokens - all of them must inherit the label
- **Long ads.** BERT only takes 512 tokens - the sliding window keeps overlap so context is preserved
- **Padding.** Labels are padded with the id of “O” here - your loss must handle this consistently

Outputs you produce: `train_dataset`, `test_dataset`, and `id2label.json` - all consumed by Task 2.

Task 2 - Training the BERT Model

Standard fine-tuning - but with class weights, gradient accumulation, and early stopping.

The training loop in five moves

1. **Load** BertForTokenClassification from *bert-base-multilingual-cased*
2. **Optimize** with AdamW + linear warmup
3. **Loss** = CrossEntropy with class weights (rare zones get more weight)
4. **Validate** every few steps - track loss on the test set
5. **Stop early** when validation loss stops improving

Things you need to fill in

- `class_weights_list` from label frequencies in the training set
- `evaluate_model()` - loss and accuracy on *active* (non-padded) tokens
- `loss.backward()` and the `optimizer.step()` / `scheduler.step()` block under gradient accumulation

Use TensorBoard from day one

Log training loss, validation loss, validation accuracy, and the learning rate. You will catch problems hours before you would have noticed them in the terminal.

Task 3 - Evaluating the Zone Model

Accuracy is misleading on imbalanced data. Use the right tool: `segeval`.

What to report

- **Per-zone** precision, recall, F1 - so you can see *which* zones the model struggles with
- **Overall** macro-average (treats every zone equally) and micro-average (token-weighted)
- Convert predicted ids back to label strings via `id2label`, drop padding, then feed to `segeval`

Then look at the actual mistakes

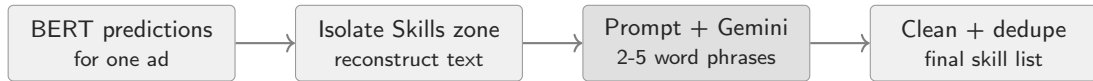
- Pick a handful of test ads where the model is wrong - read them
- Watch for confusions: *Skills vs Responsibilities*, *Benefits vs Company description*
- Watch for *boundary* errors at the start and end of zones - these are very common

Why this matters for Part 2

The skill extraction stage takes the predicted “Fähigkeiten und Inhalte” zone as its input. *Errors at this step propagate downstream.* Knowing where BERT is unreliable tells you where Gemini is being asked an unfair question.

Part 2 - From Zones to Skills

Take the “Fähigkeiten und Inhalte” text. Ask Gemini for skills. Parse. Done.



What “a skill” must look like

- A meaningful phrase, *minimum two, maximum five* words
- Examples: “strategic project management”, “cloud infrastructure deployment”, “agile software development”
- Counter-examples: “Python” (too short), “ability to work in a fast-paced agile environment” (too long)

Why constrain the format? It forces the LLM to commit to specific, comparable phrases instead of returning either single buzzwords or paragraph-long descriptions.

Prompt Engineering for Skill Extraction

The prompt is the model. Spend real time on it.

Ingredients of a good prompt for this task

- **Role** - “You are an expert HR analyst specializing in skill identification.”
- **Input framing** - tell it the text is from the Skills section of a job ad
- **Task** - extract relevant professional skills, nothing else
- **Format** - “Return a Python list of strings.”
- **Constraint** - “Each skill: minimum 2 words, maximum 5 words.”
- **Few-shot examples** - one or two input/output pairs - this alone often doubles quality

What to do with the response

- Parse the string into a real Python list - validate the structure
- Drop anything that violates the 2-5 word rule - the model will sometimes break it
- Deduplicate, store with the source ad id (CSV or JSON)

Practical: treat your API key as a secret (env variable, not hard-coded). Watch the rate limits. Add error handling for failed calls.

Task 4 - New Data via Web Crawling (*optional*)

If your pipeline works on the labeled data, prove it works on the wild web too.

What this adds to the project

- Tests the full pipeline on *contemporary, real-world* ads it has never seen
- Lets you scale up the skill database far beyond the original `annotated.json`
- A nice section in the final report - and useful skills outside this course

The pragmatic choices

- Simple, static pages: `requests` + `BeautifulSoup4`
- JavaScript-rendered or larger crawl: reach for `Scrapy`
- Clean the HTML, normalize whitespace, store one ad per record (`txt`, `csv`, or `json`)

Be a good citizen

Read the site's `robots.txt`. Add delays between requests. Do not collect personal data you do not need. The point is technical proof - not aggressive scraping.

Technical Stack and Deliverables

Standard Python NLP toolchain. Nothing exotic. Pick versions and stick to them.

Stack

- **Python 3.x**, **Pandas** for data handling
- **PyTorch** + Hugging Face **transformers** for BERT
- **seqeval** for sequence-labeling metrics
- **TensorBoard** for training monitoring
- **google-generativeai** for the Gemini API
- **requests** + **BeautifulSoup4** (or Scrapy) for crawling

What you hand in

- Source code for all five tasks - clean, commented, runnable
- `best_model.pt` and `id2label.json`
- Crawled dataset (if Task 4 done) and the final skill dataset
- A written report: motivation, methodology, results, error analysis, future work

One reflex that will save you a week

Pin your package versions in a `requirements.txt`. Hugging Face APIs change. “It worked yesterday” is not a defense.

Getting Started - The Recommended Path

Build the foundation first. Glamour later.

1. Read before you code.

Study `preprocessing.py` and `training_template.py` together with their support PDFs. You should be able to explain every step before changing anything.

2. Get a baseline running end-to-end.

Tasks 1 - 3. A model that trains and evaluates - even badly - is worth more than a perfect plan that never runs.

3. Then improve it.

Class weights, learning-rate schedule, more epochs. Use the validation curves to decide what is helping.

4. Plug in Gemini (Task 5).

Test on the original test split first - you have ground-truth zones for sanity checking the LLM's input.

5. Crawl new data (Task 4) - only if everything above works.

Optional, recommended, but a poor substitute for a working pipeline.

What You Will Walk Away With

This project is small enough to finish, big enough to teach the real skills.

- Hands-on experience fine-tuning a transformer on a real, messy, multilingual dataset
- A working sense of where neural sequence labeling helps - and where it breaks down
- Practical prompt engineering against a production LLM API
- A complete, end-to-end NLP pipeline you can put on your CV

Questions, dead ends, design decisions - bring them early.

`marcel.blattner@hslu.ch`

`https://calendly.com/marcel-btec/30min`

"All models are wrong, but some are useful." - George Box